Proximal Policy Optimization (PPO) no LunarLander-v3 — relatório experimental

Vinícius Tormin e **Bruno Calura** · Aprendizado por Reforço, UFG · orçamento de 2M passos de ambiente

Fork com `algorithms/ppo.py`, `networks/discrete_actor_critic.py` e os configs usados (`configs/ppo_lunarlander_tuned.yml`, `configs/ppo_lunarlander_targetkl.yml`, `configs/a2c_lunarlander.yml`): github.com/viniciustormin/t-zero-learning

Protocolo. `tests/test_ppo.py` 20/20 antes de qualquer treino. Cada **braço** = um config + overrides (`tools/run_sweep_ppo.py`, que lista todos), sempre nos seeds 1 e 2; cada rodada muda **uma** coisa em relação ao melhor braço anterior. O diário completo, com todos os números, está em `report/tuning_log_ppo.md`. Sem conta no W&B: `WANDB_MODE=offline`, e as figuras vêm do espelho local de cada `wandb.log` (`tools/train_csv.py` → `sweep_results_ppo/<braço>__seed<N>/metrics.jsonl`, `run = braço/seed`). Linha grossa = média dos seeds, faixa = envoltória; curvas suavizadas por EMA. Os runs rodam até 10 em paralelo (1 thread cada), então **tempo e SPS só são lidos dos runs isolados** `bench_*` (um por vez). **Atenção ao `eval`:** a avaliação final do framework são **10 episódios amostrados** da política (`deterministic=False`). Quando 1 ou 2 terminam em queda, o desvio-padrão passa de 100; por isso também reavaliamos os modelos salvos em 100 episódios (`evaluate.py --eval-episodes 100`).

Parte 4 — Resolver o LunarLander

chave	inicial → final	rodada	motivo observado (figura, run) → efeito
`ppo.learning_rate`	2.5e-4 → 1e-3	1	`base`: `approx_kl` ≈ 1e-4 e `clipfrac` = 0 do início ao fim (Fig. 1): a política quase não se move por rollout. → `r1_lr1e-3`: KL inicial 0.0076, `clipfrac` 0.04–0.05, retorno de 3/8 para 173/110
`ppo.update_epochs`	4 → 10	2	Mesmo sintoma; `r1_ep10` sozinho leva o retorno a 90/105. → `r2_lr_ep10`: os dois seeds passam de 200 em 0.29M/0.28M (Fig. 2, azul)
`total_timesteps`	1M → 2M	3	`r2_lr_ep10` **não sustenta** os 200: seed 1 cai a 34 em 0.71M, junto com picos de `approx_kl` (0.018–0.025) e a entropia voltando a subir. → `r3_2m`: tempo para se recuperar; 240/234 no fim, mas a queda continua lá (66 em 0.7M)
`num_envs`	4 → 16	3	As quedas do `r3_2m`. Batch 512 → 2048: vantagens normalizadas sobre minibatches de 512 em vez de 128. → `r3_envs16/final`: KL inicial de 0.011 para 0.007, ≥ 200 a partir de ≈1M nos dois seeds, sem a queda; SPS dobra

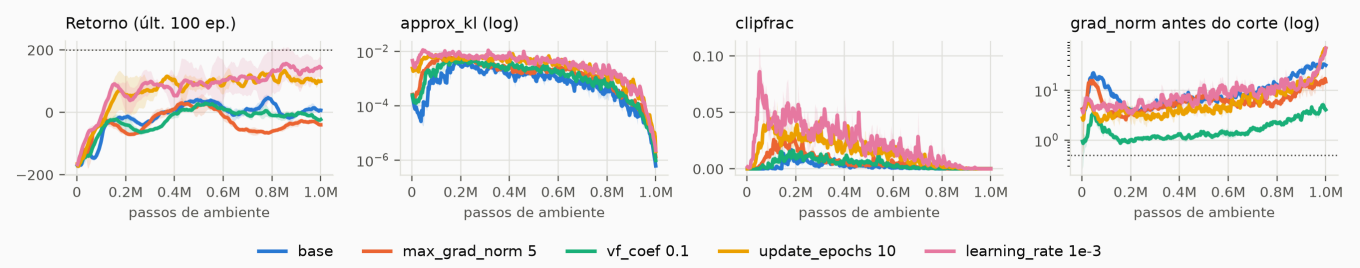


Fig. 1 — Rodada 1, uma mudança por braço sobre o `base` (1M, seeds 1–2). A linha pontilhada em `grad_norm` é o `max_grad_norm = 0.5`. Todos os braços cortam o gradiente em ≈100% dos passos; só os que movem mais a política (`lr`, `epochs`) sobem o KL e o `clipfrac` e saem do platô.

Rodada 0 → 1: a hipótese errada também conta. O `base` estaciona entre 0 e 50 a partir de 400k, com episódios de ≈500–1000 passos (paira sem pousar). A primeira leitura da Fig. 1 foi o `grad_norm`: mediana de ≈6 contra `max_grad_norm = 0.5`, cortado sempre. Com `value_loss` ≈50, apostamos que o gradiente do crítico "gastava" a norma e o corte encolhia o passo da política ≈12×. Testamos `max_grad_norm 5` e `vf_coef 0.1` (vermelho e verde na Fig. 1), e a hipótese foi **refutada**: os dois terminam abaixo do `base` (–35/–44 e –24/–19). O motivo é que `DiscreteActorCritic` tem ator e crítico em MLPs separados, então o corte apenas multiplica o gradiente do ator por uma constante, e o Adam, que divide cada parâmetro pela raiz da média móvel de g^2, desfaz essa constante. O que decide o tamanho do passo é a `learning_rate` e o número de passos de gradiente por rollout, e os dois controles (`lr 1e-3`, `ep 10`) foram os únicos que funcionaram.

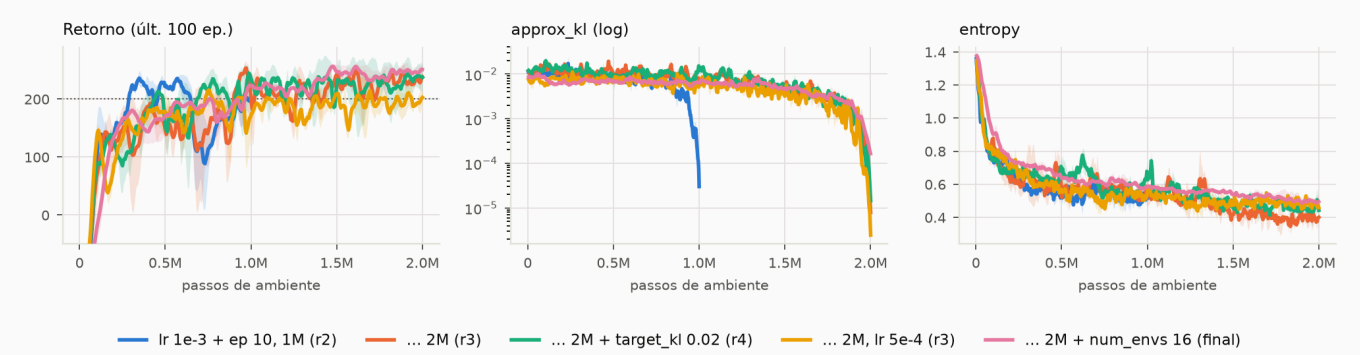


Fig. 2 — Rodadas 2–4, a partir de `lr 1e-3 + update_epochs 10`. Azul: 1M, chega a 200 primeiro e cai antes de 1M. Laranja/verde: o mesmo com 2M, sem e com `target_kl 0.02`. Amarelo: `lr 5e-4`. Rosa: `num_envs 16`, o config final.

Runs que não entraram no config final (todos com 2 seeds; números em `tuning_log_ppo.md`). **`num_minibatches 16`** (r2): o mesmo número de passos de gradiente que `ep 10`, com minibatches 4× menores; o KL é mais ruidoso e o resultado fica pior (190/155). **`gamma 0.999`** (r2 e r5): não ajuda nas duas vezes; no r5 a entropia termina **mais alta** (0.67) e o eval de 100 episódios cai para 210/229. **`lr 5e-4`** (r3): mais estável, mas lento; não chega (211/195, eval 179/185). **`target_kl 0.02`** (r4; via YAML, porque `--override ppo.target_kl=0.02` vira **string**: o parser tira o tipo do valor atual, `None`): corta só 0.4 das 10 épocas em média e o seed 2 ainda despenca de 236 para 168 (Fig. 2, verde). O limite age **depois** do minibatch que passou do alvo. **`ent_coef 0`** (r5): a entropia final é a mesma (≈0.5), ou seja, não era o bônus que mantinha a política ruidosa; sem ele, ela se fixa cedo numa política pior (155/176).

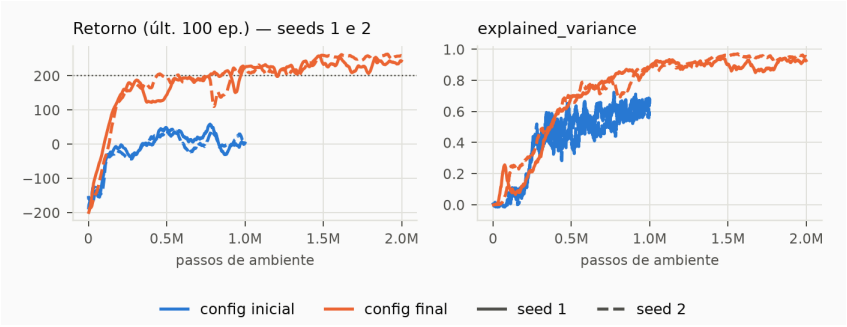


Fig. 3 — Config inicial × final, cada seed uma linha. O `explained_variance` do final chega a 0.9–0.95; o do inicial oscila em torno de 0.5.

seed	ret100	eval 10 ep.	eval 100 ep.
1	239.0	204·198·215	228 ± 97
2	257.6	283·220	242 ± 78
3	265.7	270	254 ± 81
4	244.3	269	254 ± 69

O treino é determinístico (seeds 1 e 2 de `r3_envs16`, `final` e `bench_ppo` têm curvas idênticas), mas a avaliação não é: **o mesmo modelo** do seed 1 recebeu 204.1, 197.6 e 215.0. No run `r3_envs16` os seeds 1 e 2 cumprem o critério (ret100 239/258, eval 204/283). Os seeds 3 e 4 e o eval de 100 episódios mostram que a margem real é de ≈30–50 pontos, não de 4.

Parte 5 — PPO × A2C

O A2C. É o `algorithms/a2c.py` da atividade anterior. Partimos do `a2c_cartpole` com `env_id=LunarLander-v3` e 2M passos (**HP do CartPole**: `num_envs 8`, `num_steps 5`): um seed chega a 200 em 1.04M e depois cai para 83, o outro nunca passa de 94. O ajuste é o que o piloto da atividade de A2C selecionou (`configs/a2c_lunarlander.yml`): `num_steps` 5 → 128 (decisivo: o ±100 do pouso só chega no fim do episódio), `num_envs` 8 → 16, `hidden_layers_size` 64 → 128, `vf_coef` 0.5 → 0.25, `max_grad_norm` 0.5 → 1.0, com o mesmo orçamento de 2M. Com isso o batch fica igual ao do PPO final (16 × 128 = 2048), e as duas configurações fazem as mesmas 976 iterações.

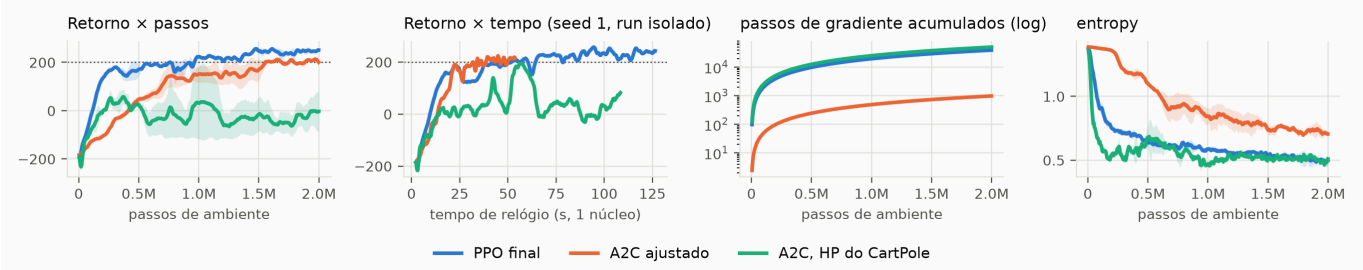


Fig. 4 — PPO final × A2C, 2M passos. Painéis 1, 3 e 4: seeds 1–2 (média e envoltória). Painel 2: seed 1 de cada, rodado sozinho na máquina (`bench_ppo`, `bench_a2c`, `bench_a2c_cp`). No A2C, os passos de gradiente são derivados (um por iteração).

	passos até 0 (M)	até 100 (M)	até 200 (M)	ret100 final	eval 100 ep.	tempo total	SPS	passos de grad.
PPO final	0.09 / 0.12	0.14 / 0.16	**0.54 / 0.42**	**239 / 258**	**228 / 242**	126 s	16.2k	39 040
A2C ajustado	0.39 / 0.43	0.65 / 0.77	1.12 / 1.55	207 / 182	200 / 190	**54 s**	**39.2k**	976
A2C, HP CartPole	0.21 / 0.14	0.74 / —	1.04 / —	83 / –89	—	108 s	18.8k	50 000

Seeds 1 / 2. Tempo e SPS: run isolado do seed 1. Tempo até 0/100/200 nesse run: PPO 9/12/37 s, A2C ajustado 13/20/31 s, A2C HP CartPole 13/41/57 s.

Eficiência amostral. O PPO chega a cada limiar com 2–4× menos passos de ambiente (0 em ≈0.1M contra ≈0.4M; 200 em 0.42–0.54M contra 1.1–1.6M). Com os mesmos 2M passos, **o PPO chega mais longe**: ret100 de 239/258 contra 207/182, eval de 100 episódios de 228/242 contra 200/190. Os seeds 3 e 4 do PPO (266/244) confirmam que isso não depende do seed.

Tempo de relógio. Por passo de ambiente, o A2C é 2.4× mais rápido (39.2k contra 16.2k SPS). Os dois coletam o mesmo batch de 2048 transições por iteração e fazem as mesmas 976 iterações; a diferença está toda em `charts/num_updates` (Fig. 4, painel 3): o A2C dá **um** passo de gradiente por rollout, e o PPO dá 10 épocas × 4 minibatches = **40**. Cada transição passa por forward/backward 10 vezes. O A2C com HP do CartPole tem o caso inverso: 50 000 passos de gradiente (um a cada 40 transições) e metade do SPS do A2C ajustado, embora cada passo seja barato, porque o custo fixo de cada atualização se repete 51× mais. **Com o mesmo orçamento de tempo, a resposta muda:** o A2C ajustado chega a 200 em 31 s e o PPO em 37 s, e no fim dos 54 s do A2C os dois estão empatados (207 contra 217). A vantagem do PPO em tempo aparece só depois disso, com o retorno subindo até ≈240 e ficando lá.

Estabilidade. Entre seeds, o PPO final fica em 239–266 (4 seeds) e o A2C ajustado em 182–207. Com os HP do CartPole, o A2C vai de 83 a –89, a maior dispersão de todas. Quedas: o PPO **não** é monotônico no começo (Fig. 3: seed 1 cai de 185 para ≈125 em 0.4M e seed 2 de ≈200 para ≈135 em 0.8M), mas a partir de ≈1M fica ≥ 200 nos 4 seeds. O A2C ajustado sobe mais devagar e com menos quedas (maior queda de 79/51 contra 89/137 do PPO), porque com um passo de gradiente por rollout a política nunca se move muito. O PPO **ganharia** essas quedas se o clipping não existisse (abaixo).

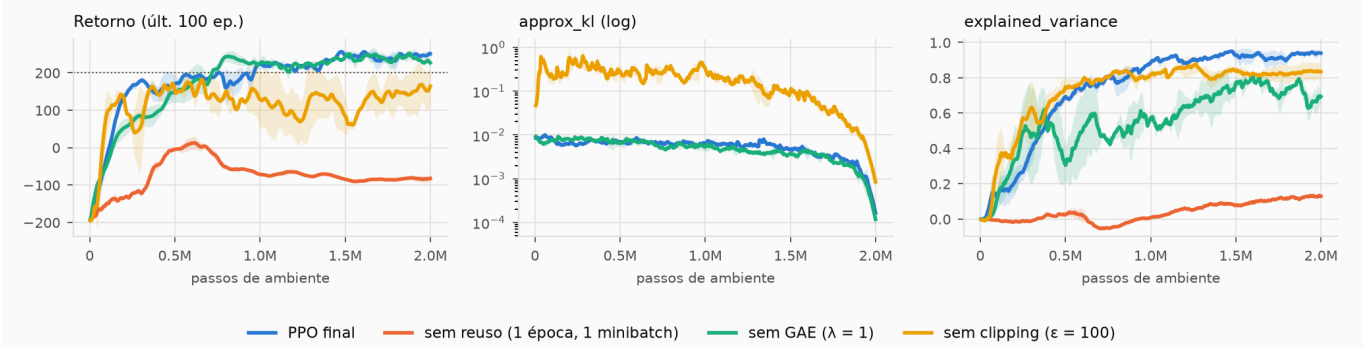


Fig. 5 — Mecanismo: o PPO final com uma das mudanças desligada (seeds 1–2, 2M). Sem reuso, com 1 época e 1 minibatch, a razão é 1 por construção; o KL medido antes do único passo é 0 e fica fora do eixo log. `clip_coef = 100` desliga também o clip do crítico (`clip_vloss`).

Mecanismo. Ablações sobre o config final (Fig. 5); a razão de probabilidades não pode ser desligada sozinha, porque só tem efeito quando há reuso. **(1) Reuso de amostras explica a eficiência amostral e o custo por passo.** Sem ele (1 época, 1 minibatch: 976 passos de gradiente, como o A2C), o PPO **colapsa**: ret100 de −75/−89, entropia parada em 0.95, `explained_variance` de 0.12. Nem o ator nem o crítico aprendem com 976 passos sobre vantagens normalizadas. Isso também explica a rodada 1 (Fig. 1: `update_epochs 10` foi uma das duas mudanças que funcionaram) e os 40× mais passos de gradiente e 2.4× menos SPS da Fig. 4. **(2) Razão + clipping explicam por que esse reuso não destrói a política.** Sem clipping, o `approx_kl` vai a 0.3–0.4 (50× o do final, Fig. 5 centro); o começo é ainda mais rápido (100 em 0.07M), mas vêm quedas de 214–257 pontos e o fim fica em 215/141 (eval 197/44). As quedas do PPO na Fig. 2 (azul, KL 0.02) são a mesma coisa em escala menor, e `num_envs 16` resolveu porque reduziu o KL por atualização. **(3) GAE explica parte da velocidade e a qualidade do crítico.** Com $\lambda = 1$ (exatamente o retorno de n passos do A2C, como o teste da Parte 1 verifica), o PPO ainda resolve (230/212), mas chega a 200 mais tarde (0.71M/0.58M contra 0.54M/0.42M), com `value_loss` 3× maior (≈310 contra ≈90) e `explained_variance` de 0.7 contra 0.9 (Fig. 5, direita): o alvo do crítico tem mais variância. É o componente de menor efeito dos três.